11.1. Hold Current Scaling

During standstill, the current can be scaled down considerably in most applications because the energy demand is lower than during motion. In addition to the scaling value, the standby delay must be configured. The delay defines the time between ramp stop and startup of hold scaling. Whenever the delay is set to 0, hold scaling is immediately enabled at the end of the velocity ramp. Because most applications require waiting for system oscillations after ramp stop, this delay must be set up in most cases.

In order to set up and enable hold current scaling, do as follows:

Action:

- Set *STDBY_DELAY* register 0x15 according to the duration of the time after ramp stop hold scaling should be enabled.
- Set *HOLD_SCALE_VAL* = *SCALE_VALUES* (31:24) according to the maximum current during motor standstill.
- Set *hold_current_scale_en* = 1 (*CURRENT_CONF* register 0x05).
- Set *closed_loop_scale_en* = 0 (*CURRENT_CONF* register 0x05).

Result:
The standby timer is started as soon as *VACTUAL* reaches 0. After *STDBY_DELAY* clock cycles the standby timer expires that activates the hold scaling phase.

Standby Status

The standby status can be forwarded via STDBY_CLK output pin.

In order to generate an output standby signal, do as follows:

Action:

- Set *stdby_clk_pin_assignment* (1) = 0 (Bit14 of *GENERAL_CONF* register 0x00).
- Set *stdby_clk_pin_assignment* (0) (Bit13 of *GENERAL_CONF* register 0x00) according to the active voltage level of the output pin.

Result:
STDBY_CLK output pin forwards the internally generated standby status. The active output level equals *stdby_clk_pin_assignment* (0).

11.2. Freewheeling

Some applications require a freewheeling behavior after ramp stop. This means that the current values are set to 0. A delay timer can be configured to define the time between standby start and the beginning of freewheeling.

In order to set up and enable freewheeling, do as follows:

Action:

- Set *FREEWHEEL_DELAY* register 0x16 according to the duration of the time after standby start, so that freewheeling is activated accordingly.
- Set *freewheeling_en* = 1 (*CURRENT_CONF* register 0x05).
- Set *closed_loop_scale_en* = 0 (*CURRENT_CONF* register 0x05).

Result:
The freewheeling timer is started as soon as the standby mode is activated. After completion of *FREEWHEEL_DELAY* clock cycles, the freewheeling timer expires that activates the freewheeling phase.

`i` Just before the velocity ramps starts internal scaling is set to the standby scaling value. This avoids starting the ramp at current values that are equal to 0.

Read entire documentation; especially the "Supplemental Directives" on page 218.

11.3. Current Scaling during Motion

If the current values need to be scaled during motion, several options are available. Up to three scaling values can be selected: Two drive scaling values and one boost scale value. Different scale values can be automatically assigned to the various sections of the velocity ramp.

11.3.1. Drive Scaling

Drive scaling is the preferred direct and mostly unconditional scaling option. If no boost scaling is enabled, the current values are scaled according to the given scale value, independent of the present ramp status.

In order to set up and enable only drive current scaling, do as follows:

Action:

- Set *DRV1_SCALE_VAL* = *SCALE_VALUES* (15:8) according to the maximum current during motion.
- Set *drive_current_scale_en* = 1 (*CURRENT_CONF* register 0x05).
- Set *closed_loop_scale_en* = 0 (*CURRENT_CONF* register 0x05).

Result:

As long as no other motion scale options are activated the current values of the MSLUT are scaled according to *DRV1_SCALE_VAL* during motion (*VACTUAL* <> 0).

11.3.2. Alternative Drive Scaling

A second drive scale parameter can be assigned in order to differentiate the motion scaling according to the internal ramp velocity.

In order to set up and enable drive current scaling with two different scaling values, do as follows:

Action:

- Set *VDRV_SCALE_LIMIT* register 0x17 [pps] according to switching velocity at which drive scaling will change.
- Set *DRV1_SCALE_VAL* = *SCALE_VALUES*(15:8) according to maximum current during motion below *VDRV_SCALE_LIMIT*.
- Set *DRV2_SCALE_VAL* = *SCALE_VALUES*(23:16) according to maximum current during motion beyond *VDRV_SCALE_LIMIT*.
- Set *drive_current_scale_en* = 1 (*CURRENT_CONF* register 0x05).
- Set *sec_drive_current_scale_en* = 1 (*CURRENT_CONF* register 0x05).
- Set *closed_loop_scale_en* = 0 (*CURRENT_CONF* register 0x05).

Result:

As long as no boost scaling is activated, the current values of the MSLUT are scaled according to *DRV1_SCALE_VAL* as long as *VACTUAL* ≤ *VDRV_SCALE_LIMIT*.

Whenever *VACTUAL* > *VDRV_SCALE_LIMIT* the current values are scaled according to *DRV2_SCALE_VAL*.

Read entire documentation; especially the "Supplemental Directives" on page 218.

11.3.3. Boost Current

In certain sections of the velocity ramp it can be useful to boost the current. Boost current can be assigned temporarily either after ramp start or during the whole acceleration/deceleration phase. All options can be selected separately; or in combination.

i All three options use the same scaling value *BOOST_SCALE_VAL*.

OPTION 1: BOOST SCALING AT RAMP START

In order to set up and enable boost current scaling within a defined time frame directly after the velocity ramp start-up, do as follows:

Action:

- Set *BOOST_TIME* register 0x18 according to the delay period at which boost current scaling is activated after a velocity ramp start.
- Set *BOOST_SCALE_VAL* = *SCALE_VALUES* (7:0) according to the maximum current during the boost phase.
- Set *boost_current_after_start_en* = 1 (*CURRENT_CONF* register 0x05).
- Set *closed_loop_scale_en* = 0 (*CURRENT_CONF* register 0x05).

Result:
After the velocity ramp start (*VACTUAL* = 0 before), boost scaling is activated according to *BOOST_SCALE_VAL*. The boost timer expires after *BOOST_TIME* clock cycles. Afterwards, any other selected scaling value is used, if active and selected.

OPTION 2: BOOST SCALING ON ACCELERATION SLOPES

In order to set up and enable boost current scaling for the acceleration phase of the velocity ramp, do as follows:

Action:

- Set *BOOST_SCALE_VAL* = *SCALE_VALUES* (7:0) according to the maximum current during the boost phase.
- Set *boost_current_on_acc_en* = 1 (*CURRENT_CONF* register 0x05).
- Set *closed_loop_scale_en* = 0 (*CURRENT_CONF* register 0x05).

Result:
As long as the absolute internal velocity |*VACTUAL*| increases, the boost scaling function is activated according to *BOOST_SCALE_VAL*. The present ramp state can be read out by the *RAMP_STATE* flag. Acceleration slopes are indicated by *RAMP_STATE* = b'01.

OPTION 3: BOOST SCALING ON DECELERATION SLOPES

In order to set up and enable boost current scaling for the deceleration phase of the velocity ramp, do as follows:

Action:

- Set *BOOST_SCALE_VAL* = *SCALE_VALUES*(7:0) according to maximum current during the boost phase.
- Set *boost_current_on_dec_en* = 1 (*CURRENT_CONF* register 0x05).
- Set *closed_loop_scale_en* = 0 (*CURRENT_CONF* register 0x05).

Result:
As long as the absolute internal velocity |*VACTUAL*| decreases, boost scaling is activated according to *BOOST_SCALE_VAL*. The present ramp state can be read out at the *RAMP_STATE* flag. Deceleration slopes are indicated by *RAMP_STATE* = b'10.

11.4. Scale Mode Transition Process Control

Transition from one scale value to the next active value can be configured as slight conversion. It is advisable to avoid abrupt scaling alterations, which can cause unwanted oscillations and/or motor stall. Three different parameters can be set to convert to higher or lower current scale values.

Transition to Hold Current Scaling

It is often required to peter out the motion (by smoothening the transition process from motion scaling to hold scaling) in order to avoid system standstill oscillations.

In order to configure a smooth transition from motion current scaling to hold current scaling, do as follows:

Action:

➢ Set *HOLD_SCALE_DELAY* register 0x19 according to the delay period after which the actual scale parameter is decreased by one step towards hold current scale value.

Result:

Immediately after the hold scaling current is activated, the actual scale parameter is decreased by one step per *HOLD_SCALE_DELAY* clock cycles until *SCALE_PARAM* = *HOLD_SCALE_VAL*.

i If *HOLD_SCALE_DELAY* = 0, the hold current scaling value *HOLD_SCALE_VAL* is assigned immediately whenever the hold current scaling is activated.

Transition to higher Motion Current Scaling

To avoid step loss – in case a higher scale value is assigned during motion – the transition from low to high current scale values can also be adapted.

In order to configure a smooth transition from a lower motion current scaling value to a higher motion current scaling value, do as follows:

Action:

➢ Set *UP_SCALE_DELAY* register 0x18 according to the delay period after which the actual scale parameter is increased by one step towards the higher current scale value.

Result:

Whenever a higher current scale value is assigned internally, the actual scale parameter is increased by one step per *UP_SCALE_DELAY* clock cycles until the assigned scale parameter is reached.

i If *UP_SCALE_DELAY* = 0, the higher current scaling value is assigned immediately whenever the corresponding current scaling phase is activated.

•→ *Description continued on next page.*

Transition to lower Motion Current Scaling

To avoid step loss or unwanted oscillations – in case a lower scale value is assigned during motion – the transition from high to low current scale values can be adapted also.

In order to configure a smooth transition from a higher motion current scaling value to a lower motion current scaling value, do as follows:

Action:

➢ Set *DRIVE_SCALE_DELAY* register 0x1A according to the delay period after which the actual scale parameter is decreased by one step towards the lower current scale value.

Result:

Whenever a lower current scale value is assigned internally, the actual scale parameter is decreased by one step per *DRIVE_SCALE_DELAY* clock cycles until the assigned scale parameter is reached.

i If *DRIVE_SCALE_DELAY* = 0, the lower current scaling value is assigned immediately whenever the corresponding current scaling phase is activated.

Two examples are provided on the following pages that illustrate how scaling modes can be used.

The scale parameter *SCALE_PARAM* is shown in combination with its related scale timers in clock cycles and in combination with the underlying velocity ramp.

Read entire documentation; especially the "Supplemental Directives" on page 218.

11.5. Current Scaling Examples

Scaling Mode Example 1

In this example, the following scale options are enabled:

- Standby scaling
- Freewheeling
- Boost scaling at start
- Boost scaling on deceleration ramps
- Drive scaling

The different scaling stages of the trapezoidal velocity ramp are shown in different colors in the **Figure A** below.
Figure B shows the internal scale parameter *SCALE_PARAM* as function of time. The scale parameter is not switched immediately whenever the scaling situations alters; because delay timers are used. A transition time between the assigned values is generated. Four transition phases are shown that are calculated as follows:

$$t_{START\_SCALE} = (BOOST\_SCALE\_VAL - HOLD\_SCALE\_VAL) \cdot UP\_SCALE\_DELAY \cdot f_{CLK}$$
$$t_{DN\_SCALE} = (BOOST\_SCALE\_VAL - DRV1\_SCALE\_VAL) \cdot DRV\_SCALE\_DELAY \cdot f_{CLK}$$
$$t_{UP\_SCALE} = (BOOST\_SCALE\_VAL - DRV1\_SCALE\_VAL) \cdot UP\_SCALE\_DELAY \cdot f_{CLK}$$
$$t_{HOLD\_SCALE} = (DRV1\_SCALE\_VAL - HOLD\_SCALE\_VAL) \cdot HOLD\_SCALE\_DELAY \cdot f_{CLK}$$

Figure C shows the different timers that are used:

- To finish boost scaling after start.
- To start standby scaling.
- To start freewheeling.

i These three delay values are directly determined by their respective register values 0x1B, 0x15, and 0x16.

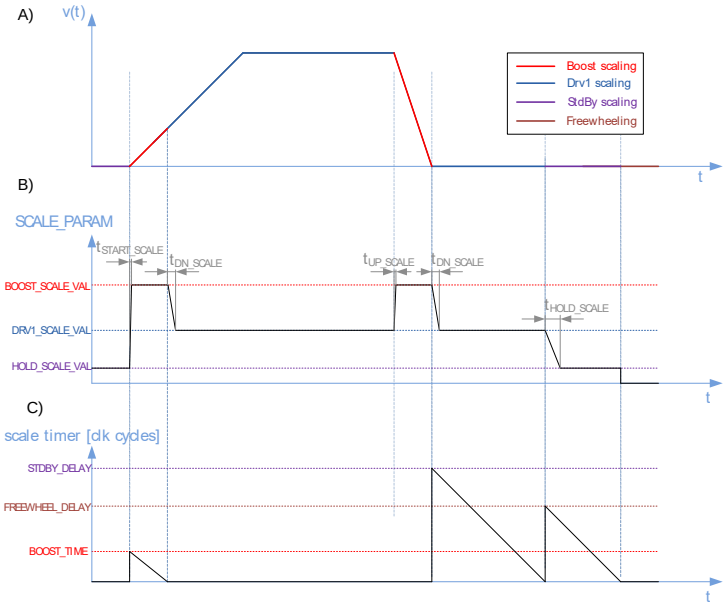


Figure 52: Scaling Example 1

Scaling Mode Example 2

In this example, the following scale options are enabled:

- Boost scaling on acceleration ramps
- Drive scaling 1 and 2

As long as |*VACTUAL*| < *VDRV_SCALE_LIMIT,* Drv1 scaling is active. Both drive scaling modes are used for the deceleration ramp because boost current is not enabled during deceleration slopes (*boost_current_on_dec* = 0).
Whenever *VACTUAL* traverses 0 the *RAMP_STATUS* switches to acceleration ramp, and boost scaling becomes enabled again.
This is shown in Figure 53 A. Figure 53 B depicts the actual scale parameter, which is altered with the formerly specified delays. In contrast to example 1, t_{START_SCALE} is changed to the following calculation:

$$t_{DN\_SCALE} = (BOOST\_SCALE\_VAL - DRV1\_SCALE\_VAL) \cdot DRV\_SCALE\_DELAY \cdot f_{CLK}$$

Whereas the other transition phases depend on whether *DRV1_SCALE_VAL* or *DRV2_SCALE_VAL* is used either; before or after the transition process.

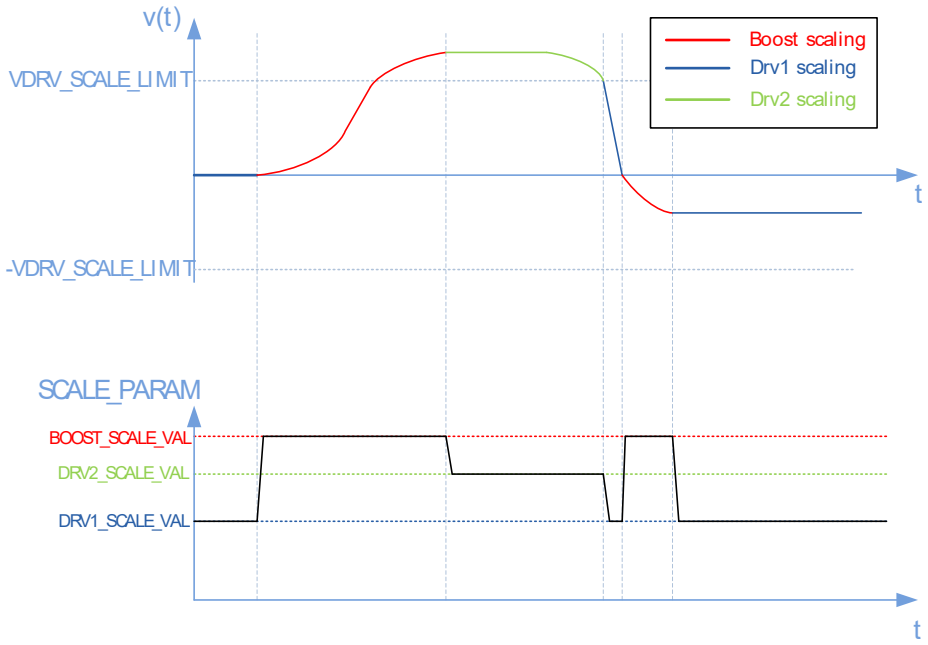


Figure 53: Scaling Example 2

Read entire documentation; especially the "Supplemental Directives" on page 218.

12. NFREEZE and Emergency Stop

In case dysfunctions at board level occur, some applications require an additional strategy to end current operations without any delay. Therefore, TMC4361 provides the low active safety pin NFREEZE.

NFREEZE Operational Principle

NFREEZE is low active. An active NFREEZE input transition from high to low level stops the current ramp immediately in a user configured way.

At the moment - *when NFREEZE switches to low* - an event *FROZEN* is triggered at *EVENTS* (10). *FROZEN* remains active until the reset of the TMC4361.

AREAS OF SPECIAL CONCERN

! It is necessary to tie NFREEZE low for at least three clock cycles because of the input filter of three consecutive sample points.

Pin Description: NFREEZE		
Pin Name	**Type**	**Remarks**
NFREEZE	Input	External enable pin; low active.

Table 48: Pin Description: NFREEZE

Pin Descriptions: DFREEZE and IFREEZE			
Register Name	**Register Address**		**Remarks**
DFREEZE	0x4E (23:0)	RW	Deceleration value in the case of an active FREEZE event.
IFREEZE	0x4E (31:24)	RW	Current scaling value in the case of an active FREEZE event.

Table 49: Pin Descriptions DFREEZE and IFREEZE

12.1.1. Configuration of FREEZE Function

Two parameters (*DFREEZE* and *IFREEZE*) are necessary in order to be able to use the TMC4361 freeze function. They are integrated in the freeze register, which can be written only once after an active reset; assuming that there has not been a ramp start before. Thus, the freeze parameters should be set directly before operation.

NOTE:

→ *Selected values cannot be altered until the next active reset. These restrictions are necessary to protect the TMC4361 freeze configuration from incorrect SPI data sent from the microcontroller in case of error.*

AREAS OF SPECIAL CONCERN

! Keep in mind that:

- The polarity of NFREEZE input cannot be assigned.
- The freeze register can always be read out.
- During freeze state ramp register values can be read out.

12.1.2. Configuration of *DFREEZE* for automatic Ramp Stop

***DFREEZE* can be used for an automatic ramp stop configuration. Two options are available:**

- **Option 1:** Use of *DFREEZE* = 0 for a hard stop.
- **Option 2:** Use of *DFREEZE* ≠ 0 for a linear deceleration ramp.

PRINCIPLE:

Due to the independence of *DFREEZE* from internal register values like *direct_acc_val_en* or the given clock frequency f_{CLK} (which can be altered by erroneous SPI signals) the deceleration value *DFREEZE* is always given as velocity value change per clock cycle. Therefore, the *DFREEZE* value is calculated as follows:

$$d\_freeze\ [pps^2] = DFREEZE / 2^{37} \cdot f_{CLK}^2$$

This leads to the same behavior of the motor and is like setting *direct_acc_val_en* to 1 for the other acceleration values during normal operation.

Configuration of *IFREEZE* current Scaling Value

***IFREEZE* can be used to configure the current scaling value during a freeze event. Two options are available:**

- **Option 1:** Use of *IFREEZE* = 0 for assigning the last specified current scaling value before the freeze event.
- **Option 2:** Use of *IFREEZE* ≠ 0 for assigning a defined current scaling value.

PRINCIPLE:

IFREEZE is a current scaling value which becomes valid in case *NFREEZE* has been tied to low and the related event (*FROZEN*) has been released.
In case *IFREEZE* is set to 0, the last scaling value before the emergency event is assigned permanently.
The scale value *IFREEZE* then manipulates the current value in the same way as explained in chapter 11, page 120.

13. Controlled PWM Output

TMC4361 offers controlled PWM (Pulse Width Modulation) signals at STPOUT and DIROUT output pins. These PWM signals can be scaled, depending on the internal velocity. If a TMC23x/24x stepper motor driver is connected and configured properly, the PWM signals are redirected to two SPI output interface pins. This avoids rerouting of signal lines at board level if SPI mode is switched to PWM mode, or vice versa.

In this chapter information is provided on the basic setup of the PWM output configuration; and also on TMC23x/24x control PWM input support.

Dedicated PWM Output Pins		
Pin Names	**Type**	**Remarks**
STPOUT_PWMA	Output	PWM output for coil A.
DIROUT_PWMB	Output	PWM output for coil B.
Connected and selected TMC23x/24x stepper motor drivers only:		
SDODRV	Output	PWM output for coil A.
NSCSDRV	Output	PWM output for coil B.

Table 50: Dedicated PWM Output Pins

Dedicated PWM Output Registers			
Register Name	**Register Address**		**Remarks**
GENERAL_CONF	0x00	RW	Bit 21: *pwm_out_en.*
CURENT_CONF	0x05	RW	*pwm_scale_en* = *CURRENT_CONF* (8): PWM scale enable switch *PWM_AMPL* = *CURRENT_CONF* (31:16): PWM amplitude at *VACTUAL* = 0.
PWM_VMAX	0x17	RW	Second assignment to *VDRV_SCALE_LIMIT*: velocity at which the PWM scale parameter reaches 1 (maximum).
PWM_FREQ	0x1F	RW	Number of clock cycles that forms one PWM period.

Table 51: Dedicated PWM Output Registers

13.1. PWM Output Generation and Scaling Possibilities

Enable PWM Output Generation

The STPOUT and DIROUT output pins generally forward internal generated microsteps and motion direction. In contrast to that, it is possible to forward the internal MSLUT value as PWM output signals, which is dependent on the PWM frequency.

In order to generate PWM output, do as follows:

Action:

- Set *PWM_FREQ* register 0x1F to the number of clock cycles for one PWM cycle.
- Set *pwm_out_en* = 1 (*GENERAL_CONF* register 0x00).

Result:

Step/Dir output is disabled and PWM signals are forwarded via STPOUT_PWMA and DIROUT_PWMB. PWM frequency f_{PWM} is calculated by:

$$f_{PWM} = f_{CLK} / PWM\_FREQ$$

If PWM Voltage mode is selected:

NOTICE

Avoid unintended overheating to prevent motor damage during PWM mode!

- **At lower velocity values PWM voltage scaling MUST be enabled.**

This will ensure smooth operation during controlled PWM mode.

PWM Duty Cycle Scaling

The duty cycle of both signals represent the sine (STPOUT) and cosine (DIROUT) values of the MSLUT.
PWM voltage scaling does not work the same way as presented for the SPI current output interface (see chapter 11, page 120). PWM scaling is adapted linearly, which depends on the internal ramp velocity. During Voltage PWM mode the scaling value at *VACTUAL* = 0 must be assigned, and also the velocity at which full scaling is reached.

In order to generate a scaled PWM output, do as follows:

Action:

- Set *PWM_AMPL* (bit31:16 of register 0x05) as start PWM scaling value.
- Set *PWM_VMAX* register 0x17 to the internal ramp velocity [pps] at which full PWM scaling is reached.
- Set *pwm_scale* = 1 (bit8 of *CURRENT_CONF* register 0x05).

Result:

- PWM_SCALE is the actual scaling value.
- In case *VACTUAL* = 0, PWM_SCALE = (*PWM_AMPL* + 1) / 2^{17}.

i Whenever the absolute velocity value increases, the scale parameter also increases linearly until it reaches the maximum of PWM_SCALE = 0.5 at *VACTUAL* = *PWM_VMAX*.

i The minimum duty cycle is calculated by DUTY_MIN = (0.5 – PWM_SCALE).

i The maximum duty cycle is calculated by DUTY_MAX = (0.5 + PWM_SCALE).

i These values set the PWM duty cycle limits of any internal ramp velocity.

13.1.1. PWM Scale Example

In *Figure* 54 below, the calculation of minimum/maximum PWM duty cycles with *PWM_AMPL* = 32767 is shown on the left side. Resulting duty cycles for different positions in the sine voltage curve are depicted on the right side. Calculated delays of minimum/maximum duty cycles are also shown.

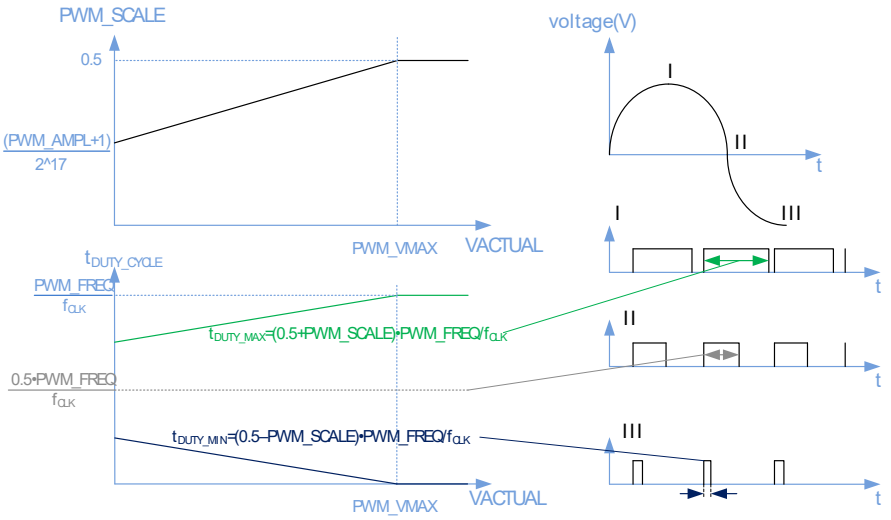


Figure 54: Calculation of PWM Duty Cycles (PWM_AMPL)

13.2. PWM Output Generation for TMC23x/24x

Controlled PWM Signals for TMC23x/24x

PWM output signals can be used for TMC23x/24x stepper motor drivers Voltage PWM mode. TMC4361 forwards the internal PWM output signals at the corresponding SPI output interface pins because the drivers share input and output pins for the SPI mode and the Voltage PWM mode. This feature enables variable operation of the TMC23x/24x in the one or the other mode without rerouting the particular signal lines at board level.

In order to generate a PWM output for TMC23x/24x stepper motor drivers, do as follows:

Action:

- Set *PWM_FREQ* register 0x1F to the number of clock cycles for one PWM cycle.
- Set *spi_output_format* = b'1000 (TMC23x) or
  spi_output_format = b'1001 (TMC24x).
- Set *pwm_out_en* = 1 (*GENERAL_CONF* register 0x00).
- Set *SPI_SWITCH_VEL* register 0x1D to 0.

Result:

- SPI output interface is disabled, controlled PWM output for TMC23x/24x is enabled.
- SDODRV_SCLK output pin forwards PWM PHA signal.
- NSCSDRV_SDO output pin forwards PWM PHB signal.
- MP2 is set to low voltage level that disables TMC23x/24x SPI mode.
- SDODRV_SCLK analyses the error flags that are forward via SDO output pin of TMC23x/24x. These error flags indicate overcurrent on any bridge or the overtemperature flag. Therefore, these three status bits of TMC4361 are altered according to the ERR flag.
- SCKDRV_NSDO is set to high voltage level to set MDBN of TMC23x/24x to high voltage level.

NOTE:

→ *Only the five pins mentioned above are set accordingly by TMC4361.*

→ *Please be aware that all other pins of TMC23x/24x must be set according to your requirements, especially ANN/MDAN = high voltage level, and INA resp. INB according to the current limit.*

i For correct hardware setup information refer to TMC23x/24x manuals.

TMC4361 with TMC23x/24x Stepper Driver

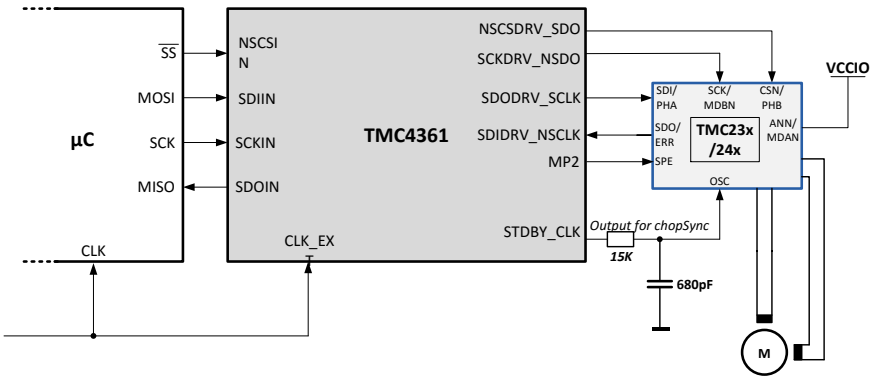


Figure 55: TMC4361 connected with TMC23x/24x operating in SPI Mode or PWM Mode

Read entire documentation; especially the "Supplemental Directives" on page 218.

13.3. Switching between SPI and Voltage PWM Modes

The hardware setup scenario, as shown on the previous page, also allows switching between SPI and Voltage PWM mode. It is advisable to enable or disable the Voltage PWM mode during standstill of the internal ramp.

In order to disable Voltage PWM mode for TMC23x/24x, do as follows:

Action:

➢ Set *pwm_out_en* = 0 (*GENERAL_CONF* register 0x00).

Result:

SPI output interface is enabled and controlled PWM output for TMC23x/24x is disabled. MP2 – that must be connected with SPE@TMC23x/24x – is set to high voltage level, which enables TMC23x/24x SPI mode.

However, it is also possible to switch between both modes during motion. Because the internal MSLUT is used either as voltage specification or as current specification, microstep loss can occur whenever the mode is switched in case the switching velocity is passed by.

i In order to overcome this, issue a microstep offset during PWM mode can be assigned.

In order to set up a TMC23x/24x configuration that switches between SPI and PWM voltage mode, do as follows:

Action:

➢ Set *PWM_FREQ* register 0x1F to the number of clock cycles for one PWM cycle.

➢ Set *pwm_out_en* = 1 (*GENERAL_CONF* register 0x00).

➢ Set *spi_output_format* = b'1000 (TMC23x) or
spi_output_format = b'1001 (TMC24x).

➢ Set *SPI_SWITCH_VEL* register 0x1D to a value [pps] at which the mode change should happen.

➢ Set *MS_OFFSET* register 0x79 (only write access) to a value between 0 and 255.

Result:

Whenever the internal velocity $|VACTUAL| < SPI\_SWITCH\_VEL$, Voltage PWM mode is activated automatically.
Whenever $|VACTUAL| \geq SPI\_SWITCH\_VEL$, SPI mode is activated automatically. During PWM mode the internal MSLUT value is modified by *MS_OFFSET*; in order to shift the resulting voltage curve of the motor coils.

Determining *MS_OFFSET*

Observing the motor coil currents with current probes is the best method for determining the required *MS_OFFSET*:

- Triggering the SPE signal will gain the switching point.
- At this point the current curves show a crack if no offset is assigned. This could lead to step loss.

i The offset can attenuate this crack to overcome this step loss.

14. dcStep Support for TMC26x or TMC2130

dcStep is an automatic commutation mode for stepper motor drivers. It allows to run the stepper with its nominal velocity, which is generated by the internal ramp generator for as long as it can cope with the motor load.

In case the motor becomes overloaded, it slows down to a lower velocity at which the motor can still drive the load. This avoids that the stepper motor stalls, and enables the stepper motor to drive heavy loads as fast as possible. Its higher torque - *available at lower velocity* – in combination with dynamic torque (from its flywheel mass) compensates mechanical torque peaks without feedback.

Dedicated dcStep Pins		
Pin Name	**Pin Type**	Remarks
MP1	Input	dcStep input signal.
MP2	Inout as Output	dcStep output signal.

Table 52: Dedicated dcStep Pins

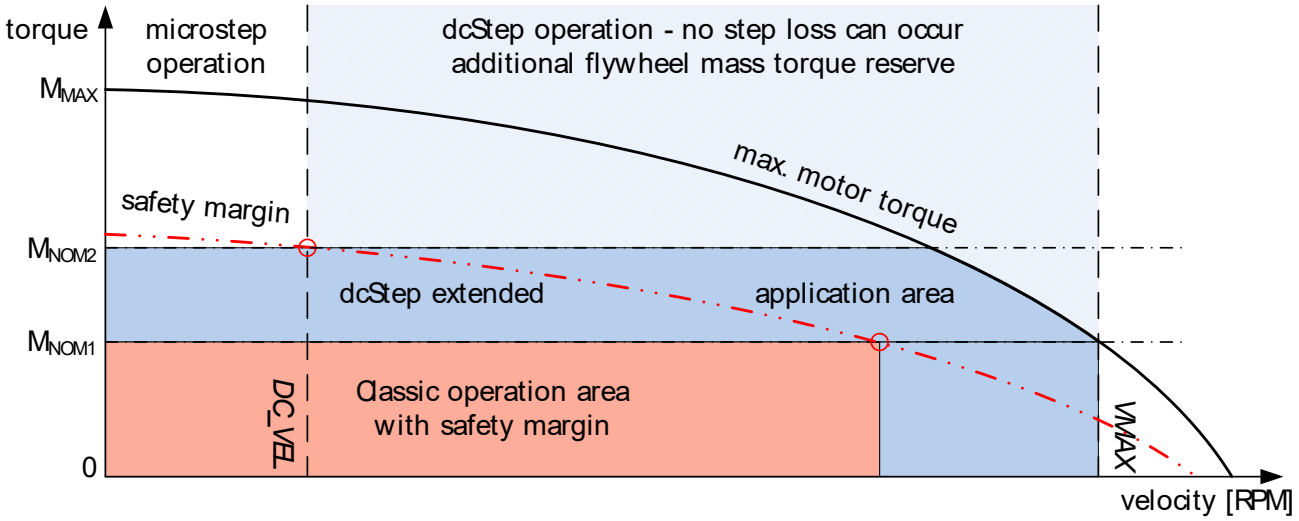
Dedicated dcStep Registers			
Register Name	**Register Address**		**Remarks**
GENERAL_CONF	0x00	RW	Bit22:21: dc_step_mode.
DC_VEL	0x60	W	Velocity at which dcStep starts (fullstep); 24 bit.
DC_TIME	0x61(7:0)	W	Upper PWM on time limit for internal dcStep calculation.
DC_SG	0x61(15:8)	W	Maximum PWM on time for step loss detection (multiplied by 16!).
DC_BLKTIME	0x61(31:16)	W	dcStep blank time after fullstep release.
DC_LSPTM	0x62	W	dcStep low speed timer; 32 bit.

Table 53: Dedicated dcStep Registers

•→Turn page for more information on how dcStep increases the usable motor torque.

dcStep increases usable Motor Torque

In a classical application, the operation area is limited by the maximum torque required at maximum application velocity. A safety margin of up to 50% torque is required, in order to compensate unforeseen load peaks, torque loss due to resonance, and aging of mechanical components. dcStep makes it possible to use the available motor torque to its fullest. Even higher short-time dynamic loads can be overcome by using motor and application flywheel mass without the danger of causing a motor stall. With dcStep, the nominal application load can be extended to a higher torque, which is only limited by the safety margin near the holding torque area (which is the highest torque the motor can provide). Additionally, maximum application velocity can be increased up to conditional maximum motor velocity.



M_{NOM}: Nominal torque required by application

M_{MAX}: Motor pull-out torque at v=0

Safety margin: Classical application operation area is limited by a certain percentage of motor pull-out torque

Figure 56: dcStep extended Application Operation Area

•→ *Turn page for more information about enabling dcStep forTMC26x stepper motor drivers.*

14.1. Enabling dcStep for TMC26x Stepper Motor Drivers

If connected to TMC26x drivers, TMC4361 must generate the dcStep signal internally; despite particular motor settings dcStep requires only very few settings, which could be tunneled via SPI through TMC4361.
dcStep directly feeds motor motion back to the ramp generator so that it becomes seamlessly integrated into the motion ramp; even if the motor becomes overloaded with respect to the target velocity. In order to set up the hardware correctly the SG_TST output pin of TMC26x must be connected to the MP1 input pin of TMC4361; and the TST_MODE pin of TMC26x must be connected to VCCIO.

i Please also refer to the corresponding TMC26x manuals for the correct motor driver settings.

In order to set up a TMC26x dcStep configuration, do as follows:

PRECONDITION: TMC26X MOTOR DRIVER SETUP:

- Set CHM = 1 (constant tOFF-Chopper).
- Set HSTRT = 0 (slow decay only).
- Set SGTO = 1 and SGT1 = 1 (on_state_xy as test signal output).
- Set TST = 1 (Test mode on).

Action:
- Set *spi_output_format* = b'1011 or b'1010 (automatic TMC26x setting).
- Set the upper PWM time *DC_TIME* slightly higher than the driver effective blank time TBL (register 0x61).
- Set *DC_BLKTIME* [clock cycles] when no comparison should happen after a fullstep release (register 0x61).
- Set *DC_SG* [clock cycles · 16] as PWM ON-time for step loss detection (0x61).
- Set *dcstep_mode* = b'01 (*GENERAL_CONF* register 0x00).

Result:
The internal dcStep at MP1 input signal approves further step generation in case the input step signals are smaller than the *DC_TIME* step length in clock cycles.

NOTE:

→ ***Even though dcStep is able to decelerate the motor during overload, stalls can occur due to certain negative influences, such as:***

The motor may stall and lose steps, e.g. because deceleration drops below obligational minimum velocity. In order to safely detect a step loss and avoid restarting of the motor, the stop on stall can be enabled
(see section 10.4.4 (page 104).
Concerning dcStep operation with TMC26x: the stall bit from the driver status is substituted by the dcStep stall detection bit.
Therefore, the first step at MP1 input directly after a step release is checked against the DC_SG value, which is the maximum PWM ON-time. In case the signal step length is smaller than DC_SG, a stall has occurred.
DC_BLKTIME specifies the number of clock cycles after a fullstep release in case nothing must be compared; because fragmented steps could occur at MP1. The first step after release that is checked is the first step after blank time. The switch to fullstep drive is performed automatically, as explained in section 10.6.4 and 10.6.5, page 111).

14.2. Setup: Minimum dcStep Velocity

dcStep requires a minimum operation velocity *DC_VEL* [pps]. *DC_VEL* must be set to the lowest operating velocity at which dcStep provides a reliable detection of motor operation. In case an overload appears, an internal dcStep signal is generated that pauses internal step generation. Because dcStep operates the motor in fullstep mode, a minimum fullstep frequency f_{FS} can be assigned.
Therefore, a dcStep low speed timer must be assigned to achieve the following minimum fullstep frequency:

$$f_{FS} = f_{CLK} / DC\_LSPTM.$$

In order to set up a minimum dcStep velocity, do as follows:

Action:

- Set the low speed timer *DC_LSPTM* register 0x62, as explained above.
- Set *DC_VEL* register 0x60 as threshold velocity value [pps] at which dcStep is activated.

Result:
Whenever the internal velocity $|VACTUAL| > DC\_VEL$, dcStep is activated, if enabled.

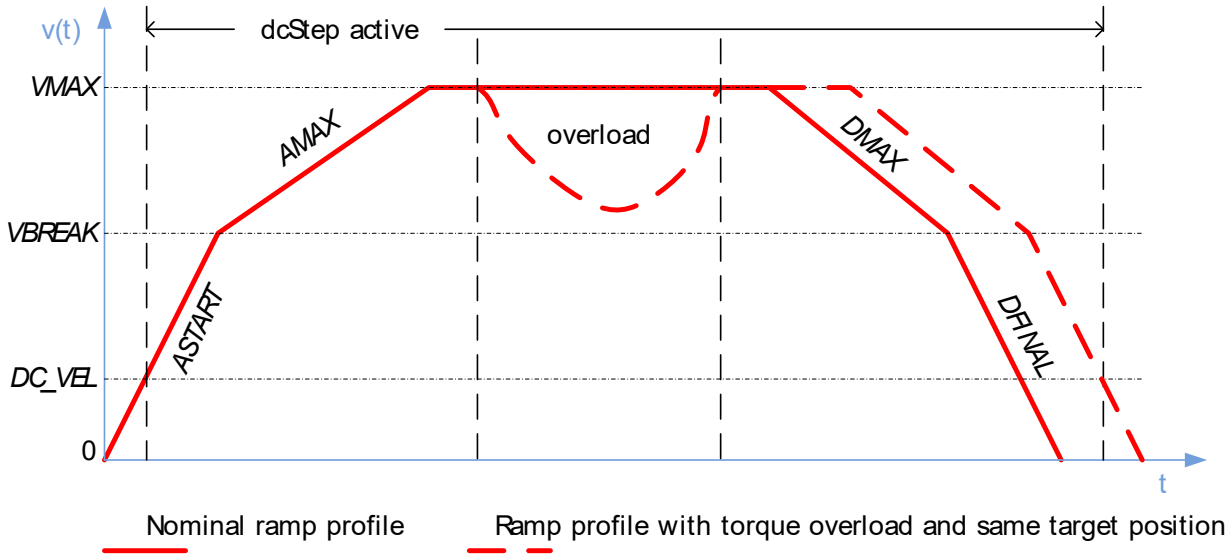


Figure 57: Velocity Profile with Impact through Overload Situation

•→*Turn Page for important information about the chopper settings for microstep and fullstep/dcStep mode.*

Read entire documentation; especially the "Supplemental Directives" on page 218.

AREAS OF SPECIAL CONCERN

Different chopper settings for microstep and fullstep/dcStep mode of TMC26x stepper driver can be transferred automatically during motion.

Switching between dcStep mode and microstep mode often requires different chopper settings for TMC26x stepper motor drivers.

It is possible to automatically transfer cover datagrams to TMC26x (see Section 10.3.6, page 101). Thereby, it is possible to switch the chopper settings of TMC26x rapidly, shortly before reaching the dcStep velocity.

NOTE:

→ *It is recommended to use this feature because dcStep requires constant OFF-time chopper settings; whereas driving with µSteps and a spreadCycle chopper provides better driving characteristics.*

In order to set up a TMC26x dcStep configuration, do as follows:

Action:

- Set the *SPI_SWITCH_VEL* register 0x1D value a little bit smaller than the *DC_VEL* register 0x60 value.
- Fill in the *COVER_LOW* 0x6C register the chopper settings for spreadCycle chopper below the *DC_VEL.*
- Fill in the *COVER_HIGH* 0x6D register the chopper settings for a constant OFF-time chopper during dcStep operation (fullstep mode).
- Set *automatic_cover* = 1 (*REFERENCE_CONF* register 0x01).

Result:

In case dcStep mode is not activated – because |*VACTUAL*| < *DC_VEL* – the spreadCycle chopper mode is activated, which is best suited for microstep operation.

In case dcStep is activated, the more suited constant OFF-time chopper mode for fullstep operation is activated.

•→ *Turn Page for more information on enabling dcStep for TMC2130 stepper motor driver.*

14.3. Enabling dcStep for TMC2130 Stepper Motor Drivers

dcStep operation with TMC2130 is similar to a handshake procedure: The MP1 input must be connected to the DCO output pin of TMC2130, whereas MP2 must be connected to the DCEN input pin of TMC2130.

In order to set up a TMC2130 dcStep configuration, do as follows:

The mandatory TMC2130 configuration MUST be executed with cover datagrams, as follows:

`i` Please refer to the TMC2130 manual for correct settings pertaining to the TMC2130 CHOPCONF and DCCTRL registers.

Action:

- Set *spi_output_format* = b'1101 or b'1100 (automatic TMC2130 setting)
- Set *dcstep_mode* = b'01 (*GENERAL_CONF* register 0x00).

Result:

In case *VACTUAL* ≥ *DC_VEL*, MP2 output is set to high voltage level to indicate that dcStep can be activated.
TMC2130 will wait for the next fullstep position to switch to dcStep operation. The dcStep signal is provided by the TMC2130 at DCO output pin.
TMC4361 is continually providing microsteps even though dcStep is enabled and activated. TMC2130 auto-generates the dcStep behavior internally.

Set up minimum dcStep/Fullstep Frequency

Because dcStep operates the motor in fullstep mode, a minimum fullstep frequency f_{FS} can be assigned. Therefore, a dcStep low speed timer must be assigned to achieve the following minimum fullstep frequency:

$$f_{FS} = f_{CLK} / DC\_LSPTM.$$

In order to set up a minimum dcStep fullstep frequency, do as follows:

Action:

- Set *DC_LSPTM* register 0x62.

Result:

After *DC_LSPTM* clock cycles expires – without lifting the internal dcStep signal – a step is enforced when dcStep is enabled.

15. Decoder Unit: Connecting ABN, SSI, or SPI Encoders correctly

TMC4361 is equipped with an encoder input interface for incremental ABN encoders, absolute SSI or SPI encoders. This chapter provides basic setup information for correct analysis of connected encoder signals.

Decoder and Closed-Loop Pins		
Pin Names	**Type**	**Remarks**
A_SCLK	Input or Output	A signal of ABN encoder or Serial Clock output for absolute SSI, or SPI encoders.
ANEG_NSCLK	Input or Output	Negated A signal of ABN encoder or Negated Serial Clock output for SSI encoder or Low active Chip Select signal for SPI encoders.
B_SDI	Input	B signal of ABN encoder or Serial Data Input of SSI, or SPI encoders.
BNEG_NSDI	Input or Output	Negated B signal of ABN encoder or Negated Serial Data Input of SSI encoders or Serial Data Output of SPI encoder.
N	Input	N signal of ABN encoder.
NNEG	Input	Negated N signal of ABN encoder.

Table 54: Dedicated Decoder Unit Pins

Decoder Unit and Closed-Loop Registers			
Register Name	**Register address**		**Remarks**
GENERAL_CONF	0x00	RW	Bit11:10: serial_enc_in_mode, Bit12: diff_enc_in_disable
INPUT_FILT_CONF	0x03	RW	Input filter configuration (SR_ENC_IN, FILT_L_ENC_IN).
ENC_IN_CONF	0x07	RW	Encoder configuration register.
ENC_IN_DATA	0x08	RW	Serial encoder input data structure.
STEP_CONF	0x0A	RW	Motor configurations.
ENC_POS	0x50	RW	Current absolute encoder position in microsteps.
ENC_LATCH	0x51	R	Latched absolute encoder position.
ENC_POS_DEV	0x52	R	Deviation between *XACTUAL* and *ENC_POS*.
ENC_CONST	0x54	R	Internally calculated encoder constant.
Encoder Register Set	0x51...58 0x62...63	W	Encoder configuration parameter.
Encoder velocity	0x65 0x66	R	Current encoder velocity (unsigned). Current filtered encoder velocity (signed).
ADDR_TO_ENC *DATA_TO_ENC*	0x68 0x69	W	Serial encoder request data.
ADDR_FROM_ENC *DATA_FROM_ENC*	0x6A 0x6B	R	Serial encoder request data response.
Encoder compensation	0x7D	W	Encoder compensation register set.

Table 55: Dedicated Decoder Unit Registers

Read entire documentation; especially the "Supplemental Directives" on page 218.

15.1.1. Selecting the correct Encoder

The encoder interface consists of six pins that can be connected with different encoder types. Depending on the encoder type, the pins serve as inputs or as outputs. If inputs are assigned, the incoming signals can be filtered, as explained in chapter 4, page 20. Consequently, *SR_ENC_IN* and *FILT_L_ENC_IN* must be set accordingly. In the following, three options are presented to select a connected encoder properly.

OPTION 1: INCREMENTAL ABN ENCODERS

In order to set up a connected incremental ABN encoder, do as follows:

Action:

➢ Set *serial_enc_in_mode* = b'00 (*GENERAL_CONF* register 0x00).

Result:
An incremental ABN encoder is selected.

OPTION 2: ABSOLUTE SSI ENCODERS

In order to set up a connected absolute SSI encoder, do as follows:

Action:

➢ Set *serial_enc_in_mode* = b'01 (*GENERAL_CONF* register 0x00).

Result:
An absolute SSI encoder is selected.

`i` In order to avoid an erroneous status of the connected absolute SSI encoder, a proper configuration is necessary prior to enabling; as described further down below on the subsequent pages: see section 15.4. on page 150.

OPTION 3: ABSOLUTE SPI ENCODERS

In order to set up a connected absolute SPI encoder:

Action:

➢ Set *serial_enc_in_mode* = b'11 (*GENERAL_CONF* register 0x00).

Result:
An absolute SPI encoder is selected.

`i` In order to avoid an erroneous status of the connected absolute SPI encoder, a proper configuration is necessary prior to enabling; as described further down below on the subsequent pages: see section 15.4. on page 150.

•→Turn page for encoder pin assignment overview.

15.1.2. Disabling digital differential Encoder Signals

If incremental ABN or absolute SSI encoders are selected, the dedicated encoder signals are treated as digital differential signals per default. For internally displaying a valid input level, the levels of a dedicated pair must be digitally inversed.

i No analog differential circuit is available.

In order to disable the digital differential input signals, do as follows:

Action:

➢ Set *diff_enc_in_disable* = 1 (*GENERAL_CONF* register 0x00).

Result:

Dedicated encoder signals are treated as single signals and every negated pin is ignored.

i Concerning absolute SPI encoders, this is done automatically.

Pin Assignment based on selected Encoder Setup						
Pin No.	**Pin Name**	**Incremental ABN**		**Absolute SSI**		**Absolute SPI**
		Differential	**Single-ended**	**Differential**	**Single-ended**	**Single-ended**
40	A_SCLK	A	A	SCLK	SCLK	SCLK
1	ANEG_NSCLK	¬A	-	¬SCLK	-	CS
10	B_SDI	B	B	SDI	SDI	SDI
11	BNEG_NSDI	¬B	-	¬SDI	-	SDO
21	N	N	N	-	-	-
22	NNEG	¬N	-	-	-	-

Table 56: Pin Assignment based on selected Encoder Setup

15.1.3. Inverting of Encoder Direction

In order to easily align the encoder direction with the motor direction it is possible to invert the encoder direction by setting one switch.

In order to invert the encoder direction, do as follows:

Action:

➢ Set *invert_direction* = 1 (*ENC_CONF* register 0x07).

Result:

The calculation of the in external position *ENC_POS* is inverted, turning increment to decrement and vice versa.

Read entire documentation; especially the "Supplemental Directives" on page 218.

15.1.4. Encoder Misalignment Compensation

If the encoder is installed correctly, the encoder values form a circle for one motor revolution. Thus, the deviation *ENC_POS_DEV* between real position *ENC_POS* und internal position *XACTUAL* forms a constant function over the whole motor revolution.
Consequently, the resulting form of a deficiently installed encoder is oval-shaped. This system failure results in a new function of *ENC_POS_DEV* that is similar to a sine function. In Figure 58 A below, the position deviation is shown as function of one motor revolution, which comprises 51200 microsteps.
TMC4361 provides an option to compensate this kind of misalignment by adding a triangular shape function that counteracts the system error. This can improve the encoder value evaluation significantly. Per default, this function is constant at 0.

In order to setup the triangular compensation function, do as follows:

Action:

- Set proper *ENC_COMP_XOFFSET* register 0x7D (15:0).
- Set proper *ENC_COMP_YOFFSET* register 0x7D (23:16).
- Set proper *ENC_COMP_AMPL* register 0x7D (31:24).

Result:
ENC_COMP_XOFFSET is 16-bit register which represents a numeral figure between 0 and 1. The resulting offset on the abscissa is calculated by:

XOFF_LOW = *ENC_COMP_XOFFSET* · microsteps/rev / 65536.

A triangular function is generated, which has its **lowest point at (XOFF_LOW; *ENC_COMP_YOFFSET*)**.
The peak is shifted at a distance of half a revolution. The **peak coordinate (XOFF_PEAK;YOFF_PEAK)** is calculated as follows:

XOFF_PEAK = *ENC_COMP_XOFFSET* · microsteps/rev / 65536 + microsteps/rev / 2.

YOFF_PEAK = *ENC_COMP_YOFFSET* + *ENC_COMP_AMPL*.

In Figure 58 A, the red line illustrates this compensation function.
Internally, the triangular function is added to the *ENC_POS* value. As a result, the position deviation is harmonized as a function of the motor revolution; which can be seen in Figure 58 B.

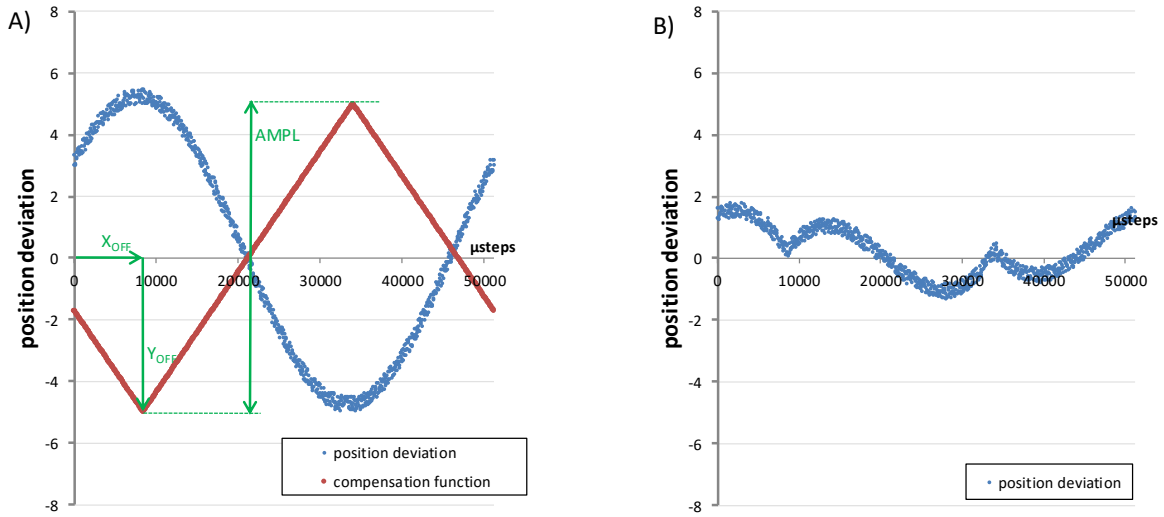


Figure 58: Triangular Function that compensates Encoder Misalignments

Read entire documentation; especially the "Supplemental Directives" on page 218.

15.2. Incremental ABN Encoder Settings

Incremental ABN encoders increment or decrement the external position counter register *ENC_POS* 0x50. This is based on A- and B-signal level transitions.

15.2.1. Automatic Constant Configuration of incremental ABN Encoder

The external position register *ENC_POS* 0x50 is based on internal microsteps. Thus, every AB transition is transferred to microsteps by a fixed constant value. TMC4361 is able to calculated this constant automatically.

In order to configure the incremental ABN encoder constant automatically, do as follows:

Action:

- Set fullstep resolution of the motor in *FS_PER_REV* (*STEP_CONF* register 0x0A).
- Set microstep resolution *MSTEP_PER_FS* (*STEP_CONF register* 0x0A).
- Set encoder resolution – the number of AB transitions during one revolution - in register *ENC_IN_RES* 0x54 (write access).

Result:

The encoder constant value *ENC_CONST* (readable at register 0x54) is calculated as follows:

$$ENC\_CONST = MSTEP\_PER\_FS \cdot FS\_PER\_REV / ENC\_IN\_RES$$

This constant is the number of microsteps through which *ENC_POS* is incremented or decremented by one AB transition.

i *ENC_CONST* consists of 15 digits and 16 decimal places.

i In case 16 bits are not sufficient for a binary representation of the decimal places, TMC4361 tries to match them to a multiple of 10000 within these 16 decimal places. Thereby, a perfect match can be achieved in case decimal representation is preferred to a binary one.

i In case the decimal representation also does not fit completely, the type of the decimal places of *ENC_CONST* can be selected manually with *ENC_IN_CONF* (0). Set *ENC_IN_CONF* (0) to 0 for binary representation; or set it to 1 for the decimal one. Keep in mind that with this approach *ENC_POS* can slightly differ from the real position; especially the further away the position moves from 0.

15.2.2. Manual Constant Configuration of Incremental ABN Encoder

For some applications it can be useful to define the encoder constant value, which in this case does not correspond to the number of microsteps per revolution; e.g. if the encoder is not mounted directly on the motor.

In order to configure the incremental ABN encoder constant manually, do as follows:

Action:

- Set *ENC_IN_RES*(31) =1.
- Set *ENC_IN_CONF*(0) to 0 for a binary or to 1 for a decimal representation as explained in the previous section.
- Set required encoder resolution in *ENC_IN_RES* (30:0) register 0x54.

Result:

ENC_CONST consists of 15 digits and 16 decimal places. The constant is the number of microsteps by which *ENC_POS* is incremented or decremented by one AB transition.

Read entire documentation; especially the "Supplemental Directives" on page 218.

15.3. Incremental Encoders: Index Signal: N resp. Z

The index signal (N or Z channel) represents a recurrence of the same position in one motor encoder revolution. TMC4361 makes use of this signal to clear the external position counter, or to take a snapshot of the external or internal position, which then can be used to refine the home position more precisely.

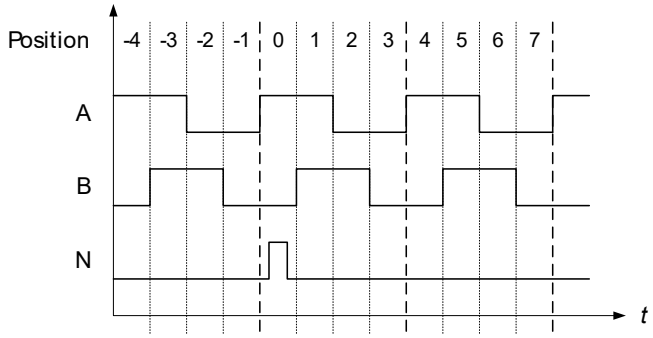


Figure 59: Outline of ABN signals of an incremental Encoder

15.3.1. Setup of Active Polarity for Index Channel

Per default, the index channel is configured low active.

In order to set up high active polarity for the index channel, do as follows:

Action:

➢ Set *pol_n* =1 (register *ENC_CONF* 0x07).

Result:

The index channel is high active.

15.3.2. Configuration of N Event

The active polarity of the index channel can be used to clear the external position counter or to take a snapshot of the external or internal position. Therefore, N event is created internally. N event is based on the active polarity of the index channel. As addition, they can also be based on the polarities of the A and B channels.

Index Channel Sensitivity

Four active polarity configuration options for the index channel are available, which are presented below. Configuration choice depends on customer-specific design wishes.

In order to set up the index channel sensitivity based on active polarity, do as follows:

Action:

➢ Set *n_chan_sensitivity* (register *ENC_CONF* 0x07) to:

Index Channel Sensitivity	
n_chan_sensitivity	**Result:**
b'00	N event is active in case index voltage level fits *pol_n.*
b'01	N event is triggered when the index channel switches to active polarity.
b'10	N event is triggered when the index channel switches to inactive polarity.
b'11	N event is triggered at both edges when the index channel switches to either active or inactive polarity.

Table 57: Index Channel Sensitivity

A and B Channel Signal Polarities for N Event

It can be useful to specify A and B channel signal polarities for N event. Per default, the polarities of both signal lines are set to 0 (low active).

In order to set up A channel polarity to high active for N event, do as follows:

Action:
- ➢ Set *pol_a_for_n* = 1 (*ENC_CONF* register 0x07).

Result:
Now, A channel signal polarity for N event is high active.

In order to set up B channel polarity to high active for N event, do as follows:

Action:
- ➢ Set *pol_b_for_n* = 1 (*ENC_CONF* register 0x07).

Result:
Now, B channel signal polarity for N event is high active.

In case A and B channel polarities do not have an influence on N event, both A and B channel polarity signals can be ignored.

In order to ignore A and B channel polarities, do as follows:

Action:
- ➢ Set *ignore_ab* = 1 (*ENC_CONF* register 0x07).

Result:
Now, the A and B channel signal polarities have no influence on N event.

15.3.3. External Position Counter *ENC_POS* Clearing

N event can be used to clear the external position register *ENC_POS* 0x50. Two choices are available: continous clearing and single clearing.

`i` Common practice is to clear to 0. However, TMC4361 offers the possibility to clear to any single microstep count.

ENC_POS Continous Clearing

In order to set *ENC_POS* on N event to continuous clearing, do as follows:

Action:
- ➢ Set *ENC_RESET_VAL* register 0x51 to the requested microstep position.
- ➢ Set *clr_latch_cont_on_n* = 1 (*ENC_CONF* register 0x07).
- ➢ Set *clear_on_n* = 1 (*ENC_CONF* register 0x07).

Result:
On every N event *ENC_POS* is set to *ENC_RESET_VAL*.

•→*Continued on next page.*

ENC_POS Single Clearing

In order to only clear *ENC_POS* for the next N event, do as follows:

Action:

- Set *ENC_RESET_VAL* register 0x51 to the requested microstep position.
- Set *clr_latch_cont_on_n* = 0 (*ENC_CONF* register 0x07).
- Set *clr_latch_once_on_n* = 1 (*ENC_CONF* register 0x07).
- Set *clear_on_n* = 1 (*ENC_CONF* register 0x07).

Result:

When the next N event occurs, *ENC_POS* is set to *ENC_RESET_VAL*. After the particular N event, *clr_latch_once_on_n* is automatically reset to 0.

15.3.4. Latching External Position

N event can be used to latch external position register *ENC_POS* 0x50 to storage register *ENC_LATCH* 0x51 (read access). Two choices are available: Continous latching and single latching.

Continous Encoder Latching

In order to continuously latch *ENC_POS* to *ENC_LATCH* on N event, do as follows:

Action:

- Set *clr_latch_cont_on_n* = 1 (*ENC_CONF* register 0x07).
- Set *latch_enc_on_n* = 1 (*ENC_CONF* register 0x07).

Result:

On every N event *ENC_POS* register 0x50 is latched to *ENC_LATCH* register 0x51.

Single Encoder Latching

In order to only latch *ENC_POS* to *ENC_LATCH* for the next N event, do as follows:

Action:

- Set *clr_latch_cont_on_n* = 0 (*ENC_CONF* register 0x07).
- Set *clr_latch_once_on_n* = 1 (*ENC_CONF* register 0x07).
- Set *latch_enc_on_n* = 1 (*ENC_CONF* register 0x07).

Result:

When the next N event occurs, *ENC_POS* register 0x50 is latched to *ENC_LATCH* register 0x51. After the particular N event, *clr_latch_once_on_n* is automatically reset to 0.

•→*For information on latching internal position turn page.*

15.3.5. Latching Internal Position

N event can be used to latch internal position register *X_ACTUAL* 0x21 to storage register *X_LATCH* 0x36 (read access). Two choices are available: Continous latching and single latching.

Continous Latching

In order to continuously latch *X_ACTUAL* to *X_LATCH* on N event, do as follows:

Action:

- Set *clr_latch_cont_on_n* = 1 (*ENC_CONF* register 0x07).
- Set *latch_enc_on_n* = 1 (*ENC_CONF* register 0x07).
- Set *latch_x_on_n* = 1 (*ENC_CONF* register 0x07).

Result:

On every N event *X_ACTUAL* register 0x21 is latched to *X_LATCH* register 0x36.

Single Latching

In order to only latch *X_ACTUAL* to *X_LATCH* for the next N event, do as follows:

Action:

- Set *clr_latch_cont_on_n* = 0 (*ENC_CONF* register 0x07).
- Set *clr_latch_once_on_n* = 1 (*ENC_CONF* register 0x07).
- Set *latch_enc_on_n* = 1 (*ENC_CONF* register 0x07).
- Set *latch_x_on_n* = 1 (*ENC_CONF* register 0x07).

Result:

When the next N event occurs, *X_ACTUAL* register 0x21 is latched to *X_LATCH* register 0x36. After the particular N event, *clr_latch_once_on_n* is automatically reset to 0.

15.4. Absolute Encoder Settings

Serial encoders provide absolute encoder angle data in contrast to step transitions, which are delivered from incremental encoders.
TMC4361 provides an external clock for the encoder in order to trigger serial data input,

15.4.1. Singleturn or Multiturn Data

TMC4361 offers singleturn and multiturn options for the serial data stream interpretation. Per default, multiturn data is not enabled. In case multiturn data is enabled, it is interpreted as unsigned count of revolutions.

In case multiturn encoder data is transmitted, do as follows:

Action:
- Set *multi_turn_in_en* = 1 (*ENC_CONF* register 0x07).
- ***OPTIONAL CONFIGURATION:*** Set *multi_turn_in_signed* = 1.
  In case multiturn data is provided as signed count of encoder revolutions.

Result:
Data from connected encoders are interpreted as multiturn data.

In case only singleturn data is transmitted TMC4361 is able to permanently calculate internally the number of encoder revolutions as if it where externally transferred multiturn data.

In case singleturn encoder data is transmitted but internally multiturn data is required, do as follows:

Action:
- Set *multi_turn_in_en* = 0 (*ENC_CONF* register 0x07).
- Set *calc_multi_turn_behav* = 1 (*ENC_CONF* register 0x07).

Result:
Data from connected singleturn encoders is internally transferred to multiturn data.

NOTE:

→ *Multiturn calculations are only correct in case two consecutive singleturn data values differ only by one step less than a half turn difference, or even less.*

Read entire documentation; especially the "Supplemental Directives" on page 218.

15.4.2. Automatic Constant Configuration of Absolute Encoder

The external position register *ENC_POS* 0x50 is based on internal microsteps. Thus, every input data angle is transferred to microsteps by a fixed constant value. TMC4361 is able to automatically calculate this constant.

In order to configure the absolute encoder constant automatically, do as follows:

Action:

- Set fullstep resolution of the motor in *FS_PER_REV* (*STEP_CONF* register 0x0A).
- Set microstep resolution *MSTEP_PER_FS* (*STEP_CONF* register 0x0A).
- Set encoder resolution in register *ENC_IN_RES* 0x54 (write access).

Result:

The encoder constant value *ENC_CONST* (readable at register 0x54) is calculated as follows:

$$ENC\_CONST = MSTEP\_PER\_FS \cdot FS\_PER\_REV / ENC\_IN\_RES$$

The external position *ENC_POS* 0x50 is calculated by multiplying the constant with the transmitted input angle.

i *ENC_CONST* consists of 15 digits and 16 decimal places.

i In contrast to incremental ABN encoders, *ENC_CONST* is always represented as binary constant.

15.4.3. Manual Constant Configuration of Incremental ABN Encoder

For some applications it can be useful to define the encoder constant value, which in this case does not correspond to the number of microsteps per revolution; e.g. if the encoder is not mounted directly on the motor.

In order to configure the absolute encoder constant manually, do as follows:

Action:

- Set *ENC_IN_RES* (31) =1.
- Set required encoder resolution in *ENC_IN_RES* (30:0) register 0x54.

Result:

ENC_CONST consists of 15 digits and 16 decimal places. The external position *ENC_POS* 0x50 is calculated by multiplying the constant with the transmitted input angle.

15.4.4. Absolute Encoder Data Setup

Encoder Data must be maintained correctly. Consequently, certain settings must be configured so that TMC4361 displays them as specified.

In order to configure absolute encoder data, do as follows:

Action:

➢ Set *SINGLE_TURN_RES* (*ENC_IN_DATA* register 0x08) to the number of singleturn data bits −1.

OPTION A1: IF MULTITURN DATA IS TRANSMITTED

➢ Set *MULTI_TURN_RES* (*ENC_IN_DATA* register 0x08) to the number of multiturn data bits −1.

OR OPTION A2: IF MULTITURN DATA IS NOT TRANSMITTED

➢ Set *MULTI_TURN_RES* = 0 (*ENC_IN_DATA* register 0x08).

➢ Set *STATUS_BIT_CNT* (*ENC_IN_DATA* register 0x08) to the number of status bits.

OPTION B1: IF STATUS FLAGS ARE ORDERED IN FRONT

➢ Set *left_aligned_data* = 0 (*ENC_IN_CONF* register 0x07).

OR OPTION B2: IF STATUS FLAGS ARE ORDERED IN FRONT

➢ Set *left_aligned_data* = 1 (*ENC_IN_CONF* register 0x07).

Result:

SINGLE_TURN_RES defines the most significant bit (MSB) of the angle data bits, whereas *MULTI_TURN_RES* defines the MSB of the revolution counter bits. Up to three status bits can be received. The number of transferred clock bits that are sent to the encoder is calculated as follows:

#SCLK Cycles= (*SINGLE_TURN_RES*+1) + (*MULTI_TURN_RES*+1) + *STATUS_BIT_CNT*

Also, the order in which the status bits occur in one encoder data stream can be configured. In Figure 59 on the next page, example setups are depicted.

NOTE:

→ *In case more than three status bits or additional fill bits are sent from the encoder, clock errors can occur because the number of transferred clock bits does not fit.*

→ *In order to prevent clock failures, MULTI_TURN_RES can be set to a higher value than otherwise required; even if the encoder does not provide multiturn data. This can result in erroneous multiturn data, which can be corrected by setting multi_turn_in_en=0 in order to skip multiturn data automatically.*

→ *In order to compensate unavailable multiturn data make use of calc_multi_turn_behav, as explained in section 15.4.1 on page 150.*

•→*Turn page for serial data output examples.*

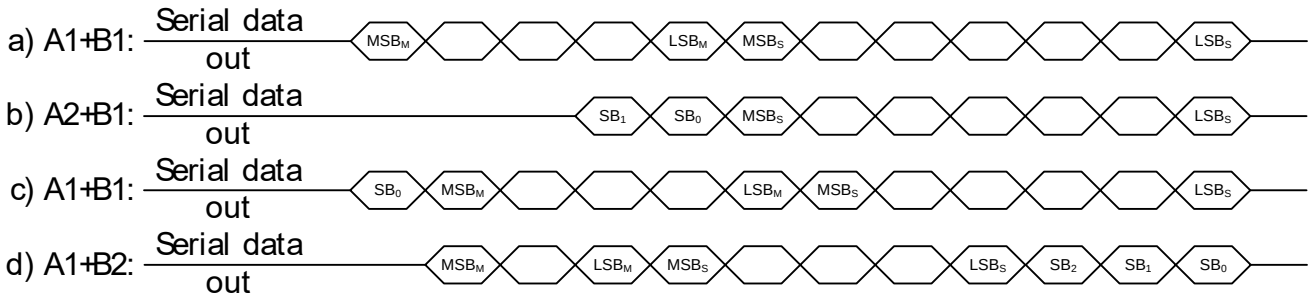


Figure 60:Serial Data Output: Four Examples

Key:

a) SINGLE_TURN_RES=6; MULTI_TURN_RES=4; STATUS_BIT_CNT=0; left_aligned_data=0

b) SINGLE_TURN_RES=6; MULTI_TURN_RES=0; STATUS_BIT_CNT=2; left_aligned_data=0

c) SINGLE_TURN_RES=5; MULTI_TURN_RES=4; STATUS_BIT_CNT=1; left_aligned_data=0

d) SINGLE_TURN_RES=4; MULTI_TURN_RES=2; STATUS_BIT_CNT=3; left_aligned_data=1

15.4.5. Emitting Encoder Data Variation

For some applications it can be useful to limit the difference between two consecutive encoder data values; for instance, if encoder data lines are subject to too much noise.
Per default, encoder data values can show a difference of 1/8th per encoder revolution, only if the limitation is enabled. The difference can be configured to a smaller value, if necessary.

In order to enable and configure encoder data variation limitation, do as follows:

Action:

- ***OPTIONAL:*** Set proper *SER_ENC_VARIATION* register 0x63 (7:0).
- Set *serial_enc_variation_limit* =1 (*ENC_IN_CONF* register 0x07).

Result:
The encoder data value that is received subsequently must not exceed the previous data more than:

Maximum tolerated deviation = *SER_ENC_VARIATION* / 256 · 1/8 · *ENC_IN_RES.*

In case the variation exceeds the above mentioned limit, the new data value is rejected internally and the status flag *SER_ENC_DATA_FAIL* is raised.

i In case *SER_ENC_VARIATION* = 0, the limit is defined by 1/8 · *ENC_IN_RES.*

Read entire documentation; especially the "Supplemental Directives" on page 218.

15.4.6. SSI Clock Generation

In order to receive encoder data from the absolute encoder, TMC4361 generates clock patterns according to SSI standard. Data transfer is initiated by switching the clock line SCLK from high to low level. The transfer starts with the next rising edge of SCLK. The number of emitted clock cycles depends on the expected data width, as explained in section 15.4.4.

Configuration Details

One clock cycle has a high and a low phase, which can be defined separately according to internal clock cycles. Per default, sample points of serial data are set at the falling edges of SCLK. Some encoders need more clock cycles – than are available during the low clock phase – in order to prepare data for transfer. Also, due to long wires, data transfer can take more time. To counteract the above mentioned issues, the delay time *SSI_IN_CLK_DELAY* (default value equals 0) for compensation can be specified in order to prolong the sampling start. Therefore, this delay configuration can automatically generate more clock cycles.
After a data request – when all clock cycles have been emitted – the serial clock must remain idle for a certain interval before the next request is automatically initiated. This interval *SER_PTIME* can also be configured in internal clock cycles.

`i` According to SSI standard, select an interval that is longer than 21 µs.

In order to configure the SSI clock generation, do as follows:

Action:

- Set *SINGLE_TURN_RES* (*ENC_IN_DATA* register 0x08) to the number of singleturn data bits −1.
- Set *MULTI_TURN_RES* (*ENC_IN_DATA* register 0x08) to the number of multiturn data bits −1 in case multiturn data is enabled and used.
- Set *STATUS_BIT_CNT* (*ENC_IN_DATA* reg. 0x08) to the number of status bits.
- Set proper *left_aligned_data* (*ENC_IN_CONF* register 0x07).
- Set proper *SER_CLK_IN_LOW* (register 0x56) in internal clock cycles.
- Set proper *SER_CLK_IN_HIGH* (register 0x56) in internal clock cycles.
- ***OPTIONAL CONFIG:*** Set proper *SSI_IN_CLK_DELAY* (register 0x57) in internal clock cycles.
- ***OPTIONAL CONFIG:*** Set proper *SER_PTIME* (reg. 0x58) in internal clk cycles.
- **Finally, set *serial_enc_in_mode* = b'01**.

Result:

TMC4361 emits serial clock streams at SCLK in order to receive absolute encoder data at SDI. If *SSI_IN_CLK_DELAY* > 0, the SDI sample points are delayed (see figures below). *SER_PTIME* defines the interval between two consecutive data requests.

`i` If differential encoder is selected, the negated clock emits at ¬SCLK; and ¬SDI is also evaluated.

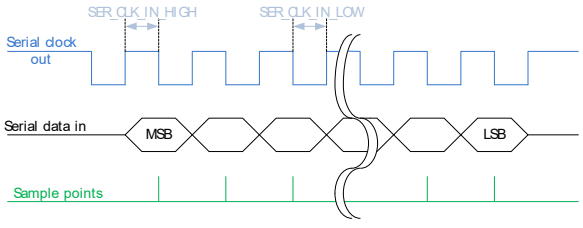


Figure 61: SSI: SSI_IN_CLK_DELAY=0

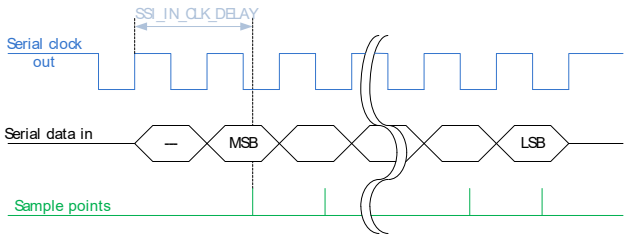


Figure 62: SSI: SSI_IN_CLK_DELAY>SER_CLK_IN_HIGH

Read entire documentation; especially the "Supplemental Directives" on page 218.

15.4.7. Enabling Multicycle SSI request

If safe transmission must be determined, it is possible to send a second request so that the encoder repeats the same encoder data. Therefore, a second interval *SSI_WTIME* must be defined.

`i` According to SSI standard, select an interval that is shorter than 19 µs.

In order to enable multicycle requests, do as follows:

Action:

- Set *ssi_multi_cycle_data* =1 (*ENC_IN_CONF* register 0x07).
- Set proper *SSI_WTIME* (register 0x57) in internal clk cycles.

Result:

After a data request – when all clock cycles have been emitted – the serial clock remains idle for *SSI_WTIME* clock cycles. Afterwards, the second request is automatically initiated to receive the same encoder data. If the second encoder data differs from the first one, error flag *MULTI_CYCLE_FAIL* (register 0x0F) and error event *SER_ENC_DATA_FAIL* (register 0x0E) is generated.
After the second data request, the next interval lasts *SER_PTIME* clock cycles to request new encoder data.

15.4.8. Gray-encoded SSI Data Streams

Several but not all SSI encoders emit angle data, which is gray-encoded. TMC4361 is able to decode this data automatically.

In order to enable gray-encoded angle data, do as follows:

Action:

- Set *ssi_gray_code_en* =1 (*ENC_IN_CONF* register 0x07).

Result:

Encoder data is recognized as gray-encoded and thus also decoded accordingly.

15.4.9. SPI Encoder Data Evaluation

SPI encoder interfaces typically consist of four signal lines. In addition to SSI encoder signal lines (SCLK, MISO), a chip select line (CS) and a data input (MOSI) to the master is provided.

SPI Encoder Communication Process

The number of bits per transfer is calculated automatically; based on proper *multi_turn_in_en*, *SINGLE_TURN_RES*, *MULTI_TURN_RES*, and *STATUS_BIT_CNT*, as explained in sections 15.4.1 (page 150) and 15.4.4 (page 152).

A typical SPI communication process responds to any SPI data transfer request when the next transmission occurs. When TMC4361 receives an answer from the encoder, it calculates *ENC_POS* immediately. The encoder slave does not send any data without receiving a request first.
Therefore, TMC4361 always sends *ADDR_TO_ENC* value to request encoder data from the SPI encoder slave device. The LSB of the serial data output is *ADDR_TO_ENC* (0).
Received encoder data is stored in *ADDR_FROM_ENC*. Thus, encoder values can be verified and compared to microcontroller data later on.

i The clock generation works similarly to SSI clock generation, as described in section 15.4.5 on page 154; based on proper *SER_CLK_IN_HIGH, SER_CLK_IN_LOW,* and *SER_PTIME*.

In order to configure a basic SPI communication procedure, do as follows:

Action:

- Set *SINGLE_TURN_RES* (*ENC_IN_DATA* register 0x08) to the number of singleturn data bits −1.
- Set *MULTI_TURN_RES* (*ENC_IN_DATA* register 0x08) to the number of multiturn data bits −1 in case multiturn data is enabled and used.
- Set *STATUS_BIT_CNT* (*ENC_IN_DATA* register 0x08) to the number of status bits.
- Set proper *left_aligned_data* (*ENC_IN_CONF* register 0x07).
- **Set correct SPI transfer mode that is described in the next section**.
- Set *ADDR_TO_ENC* register 0x68 to the specified SPI encoder address that contains angle data.
- Set proper *SER_CLK_IN_LOW* (register 0x56) in internal clock cycles.
- Set proper *SER_CLK_IN_HIGH* (register 0x56) in internal clock cycles.
- ***OPTIONAL CONFIG:*** Set proper *SER_PTIME* (register 0x58) in internal clk cycles.
- **Finally, set *serial_enc_in_mode* = b'11**.

Result:
TMC4361 emits serial clock streams at SCLK in order to receive absolute encoder data at SDI pin. The number of generated clock cycles depends on *SINGLE_TURN_RES*, *MULTI_TURN_RES*, and *STATUS_BIT_CNT*.
Pin ANEG_NSCLK functions as negated chip select line for the SPI encoder that is generated according to the serial clock and the selected SPI mode; which is described in the next section.
Pin BNEG_NSDI is the MOSI line that transfers SPI datagrams to the SPI encoder. Datagrams, which are transferred permanently to receive angle data, consists of *ADDR_TO_ENC* data.
SER_PTIME defines the interval between two consecutive data requests.

•→Turn page for information on SPI mode selection.

15.4.10. SPI Encoder Mode Selection

Per default, SPI encoder data transfer is managed in the same way as the communication between microcontroller and TMC4361. TMC4361 supports all four SPI modes with proper setting of switches *spi_low_before_cs* and *spi_data_on_cs*.

THE PROCESS IS AS FOLLOWS:

By setting ***spi_low_before_cs* = 0**, negated chip select line at ANEG_NSCLK is switched to active low **before** the serial clock line SCLK switches.

By setting ***spi_low_before_cs* = 1**, negated chip select line at ANEG_NSCLK is switched to active low **after** the serial clock line SCLK switches.

By setting ***spi_data_on_cs* = 0**, the first data bit at BNEG_NSDI is changed at the same time as the first slope of the serial clock SCLK.

By setting ***spi_data_on_cs* = 1**, the first data bit at BNEG_NSDI is changed at the same time as the negated chip select signal at BNEG_NSDI switches to active level.

In the table below, all four SPI modes are presented.

Per default, the delay between serial clock line and negated chip select line has a time frame of either *SER_CLK_IN_HIGH* or *SER_CLK_IN_LOW* clock cycles, which depends on the actual voltage level of the serial clock.
This particular interval does not always match the encoder behavior perfectly. Therefore, both the first and last intervals between the serial clock line and the negated chip select line can be specified separately in clock cycles at *SSI_IN_CLK_DELAY* register 0x57.

Below, the *SSI_IN_CLK_DELAY* interval is highlighted in red in all four diagrams.

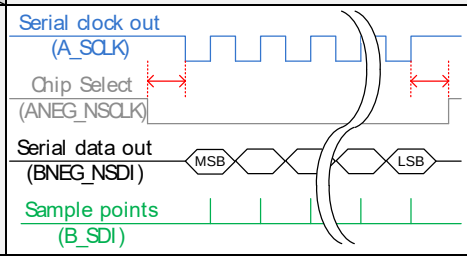
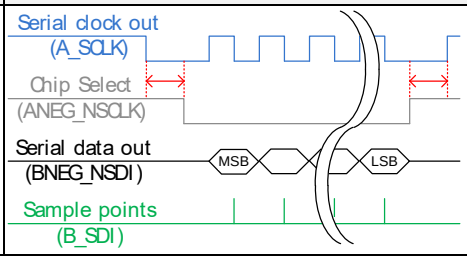
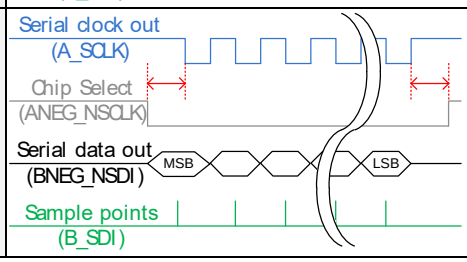
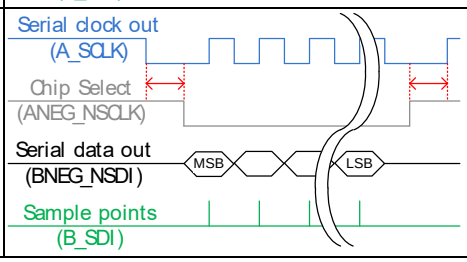
Supported SPI Encoder Data Transfer Modes		
spi_low_before_cs: / **spi_data_on_cs**	**0**	**1**
0		
1		

Table 58: Supported SPI Encoder Data Transfer Modes

15.4.11. SPI Encoder Configuration via TMC4361

Connected SPI encoder can be configured via TMC4361., which renders a connection between microcontroller and encoder unnecessary.

SPI Encoder Configuration Communication Process

A configuration request is sent using the settings of *SERIAL_ADDR_BITS* and *SERIAL_DATA_BITS,* which define the transferring bit numbers.

In order to prepare SPI encoder configuration procedures, do as follows:

Action:

- Set *SERIAL_ADDR_BITS* (*ENC_IN_DATA* register 0x08) to the number of address bits of any SPI encoder configuration datagram.
- Set *SERIAL_DATA_BITS* (*ENC_IN_DATA* register 0x08) to the number of data bits of any SPI encoder configuration datagram.

Result:
In case configuration data is transferred to the SPI encoder, *SERIAL_ADDR_BITS* bits and *SERIAL_DATA_BITS* bits are sent in two SPI configuration datagrams; exactly in this order.

Because encoder data requests occur as an endless stream, it is necessary to interrupt data requests when a configuration request occurs. Consequently, a handshake behavior is implemented.

In order to transfer configuration data to the SPI encoder, do as follows:

Action:

- Set *DATA_TO_ENC* register 0x69 to any value.
- Set *ADDR_TO_ENC* register 0x68 to the configuration address of the SPI encoder.
- Set *DATA_TO_ENC* register 0x69 to the configuration data of the SPI encoder.

Result:
The first *DATA_TO_ENC* access stops the repetitive encoder data request.
After the second *DATA_TO_ENC* access, three datagrams are sent to SPI encoder:

1. One address datagram is transmitted, which contains the *ADDR_TO_ENC* value. Data that is received simultaneously with the request is not stored.
2. One data datagram is transmitted that contains the *DATA_TO_ENC* value. Data that is received simultaneously with the request is stored in *ADDR_FROM_ENC* register 0x6A because this is the response of the *ADDR_TO_ENC* request.
3. One no-operation datagram (NOP) is transmitted. Data that is received simultaneously with the request is stored in *DATA_FROM_ENC* register 0x6B because this is the response of the *DATA_TO_ENC* request.

In order to finalize the configuration procedure and continue with the encoder data requests, do as follows:

- Read out *ADDR_FROM_ENC* register 0x6A first.
- Set *ADDR_TO_ENC* register 0x68 to the specified SPI encoder address that contains angle data.
- **Obligatory at finalization: Read out *DATA_FROM_ENC* register 0x6B.**

Result:
The configuration request data is read out. After *DATA_FROM_ENC* register readout, the encoder data request stream of angle data continues.

Read entire documentation; especially the "Supplemental Directives" on page 218.

16. Possible Regulation Options with Encoder Feedback

Beyond simple feedback monitoring, encoder feedback can be used for controlling motion controller outputs in such a way that the internal actual position matches or follows the real position *ENC_POS*. Two options are provided: PID control and closed-loop operation.
Closed-loop operation is preferable if the encoder is mounted directly on the back of the motor and position data is evaluated precisely. PID control is preferable if the encoder is located on the drive side with no fixed connection between motor and drive side; e.g. belt drives.

Closed-Loop and PID Registers			
Register Name	**Register address**		**Remarks**
ENC_IN_CONF	0x07	RW	Encoder configuration register: Closed-Loop configuration switches.
CL_TR_TOLERANCE	0x51	R	Absolute tolerated deviation to trigger TARGET_REACHED during regulation.
ENC_POS_DEV	0x52	R	Deviation between *XACTUAL* and *ENC_POS*.
Closed-Loop and PID Register Set	0x59...5F 0x60...61	W	Closed-Loop and PID configuration parameters.
Encoder velocity configuration	0x63	W	Encoder velocity filter configuration parameters.
Encoder velocity	0x65 0x66	R	Current encoder velocity (unsigned). Current filtered encoder velocity (signed).

Table 59: Dedicated Closed-Loop and PID Registers

16.1. Feedback Monitoring

Based on the difference *ENC_POS_DEV* (readout at register 0x52) between internal position *XACTUAL* and external position *ENC_POS*, a status flag *ENC_FAIL_F* and a corresponding error event *ENC_FAIL* is generated automatically.

In order to set a tolerated position mismatch, do as follows:

Action:

➢ Set *ENC_POS_DEV_TOL* register 0x53 to the maximum microstep value that represents no mismatch failure.

Result:

In case $|ENC\_POS\_DEV| \leq ENC\_POS\_DEV\_TOL$, no encoder failure flag is set.
In case $|ENC\_POS\_DEV| > ENC\_POS\_DEV\_TOL$, *ENC_FAIL_F*lag is set.

i At this point, the corresponding encoder event *ENC_FAIL* is also triggered.

16.1.1. Target-Reached during Regulation

In case one of the regulation modes is selected, TARGET_REACHED event and status flag is only released when:

$$XACTUAL = XTARGET \quad \text{and} \quad |ENC\_POS\_DEV| \leq CL\_TR\_TOLERANCE.$$

Consequently, *CL_TR_TOLERANCE* register 0x52 (only write access) is the maximal tolerated position mismatch for target reached status.

16.2. PID-based Control of *XACTUAL*

Based on a position difference error *PID_E = XACTUAL – ENC_POS* the PID (proportional integral differential) controller calculates a signed velocity value (v_{PID}), which is used for minimizing the position error. During this process, TMC4361 moves with v_{PID} until $|PID\_E| - PID\_TOLERANCE \leq 0$ is reached and the position error is removed.

v_{PID} is calculated by:

$$v_{PID} = \frac{PID\_P}{256} \cdot PID\_E \cdot \left[\frac{1}{s}\right] + \frac{PID\_I}{256} \cdot \int_0^t PID\_E \cdot dt + PID\_D \cdot PID\_E \cdot \frac{d}{dt}$$

$$v_{PID} = \frac{PID\_P}{256} \cdot PID\_E \cdot \left[\frac{1}{s}\right] + \frac{PID\_I}{256} \cdot PID\_ISUM + PID\_D \cdot PID\_E \cdot \frac{d}{dt}$$

$$v_{PID} = \frac{PID\_P}{256} \cdot PID\_E \cdot \left[\frac{1}{s}\right] + \frac{PID\_I}{256} \cdot PID\_E \cdot \frac{f_{CLK}}{128} + PID\_D \cdot PID\_E \cdot \frac{d}{dt}$$

Key:

PID_P = proportional term; PID_I = integral term; PID_D = derivate term

16.2.1. PID Readout Parameters

The following parameters can be read out during PID operation.

PID_VEL 0x5A Actual PID output velocity.

PID_E 0x5D Actual PID position deviation between *XACTUAL* and *ENC_POS*.

PID_ISUM 0x5B Actual PID integrator sum (update frequency: $f_{CLK}/128$), which is calculated by:

$$PID\_ISUM = PID\_E \cdot f_{CLK} / 128$$

•→*Turn page for information on configuration of PID regulation.*